

Capability Diagnostics of Enterprise Service Architectures using a dedicated Software Architecture Reference Model

Alfred Zimmermann

Reutlingen University
SOA Innovation Lab, Germany
alfred.zimmermann@reutlingen-university.de

Hans-Jürgen Groß

Daimler - Stuttgart
SOA Innovation Lab, Germany
hans-juergen.gross@daimler.com

Gunther Piller

University of Applied Sciences Mainz
SOA Innovation Lab, Germany
gunther.piller@fh-mainz.de

Helge Buckow

McKinsey & Company - Berlin
SOA Innovation Lab, Germany
helge_buckow@mckinsey.com

Oliver F. Nandico

Capgemini - Munich
SOA Innovation Lab, Germany
oliver.f.nandico@capgemini.com

Karl Prott

Capgemini - Hamburg
SOA Innovation Lab, Germany
karl.prott@capgemini.com

Abstract: The SOA Innovation Lab - an innovation network of industry leaders in Germany and Europe - investigates the practical use of vendor platforms in a service-oriented context. For this purpose an original service-oriented ESA-Enterprise-Software-Architecture-Reference-Model, an associated ESA-Architecture-Maturity-Framework and an ESA-Pattern-Language for supporting architecture evaluation and optimization has been researched, leveraging and extending CMMI and TOGAF, as well as other service-oriented state-of-the-art frameworks and methods. Current approaches for assessing architecture quality and maturity of service-oriented enterprise software architectures are rarely validated and were intuitively developed, having sparse reference model, metamodel or pattern foundations. This is a real problem because enterprise and software architects should know how advanced architecture quality concepts can successfully be used and how a stable foundation for introducing service-oriented enterprise architectures for adaptive systems looks like. Our idea and contribution is to extend existing enterprise and software architecture reference models and maturity frameworks to accord with a sound metamodel approach. We have developed and are presenting the idea of a pattern language for assessing the architecture quality of adaptable service-oriented enterprise systems. Our approach is based on ESARC – an Enterprise Software Architecture Reference Model we have developed.

Architecture Quality Diagnostics; Evaluation; SOA Vendor Platforms; Enterprise-Software-Architecture-Reference-Model; ESARC; SOAMMI; ESA-Patterns; Validation

I. INTRODUCTION

The growing complexity of IT landscapes is a challenge for many companies. A large number of packaged solutions platforms - mostly extended and modified - individual software solutions, legacy applications, and different infrastructure components lead to high cost and limited ability to respond quickly to new business requirements. Many companies start initiatives of service-oriented enterprise architecture management (EAM) to address this problem.

The hypothesis of our research integrates the following original ideas and generates a relevant basis for high software technology innovation impacts for practice:

1. The structure of ESARC substantiates the TOGAF standard [1] and provides for the evaluation of service-oriented enterprise software architectures a useful mapping foundation of an abstract reference structure of major architecture artifacts and their main relationships.
2. The Capability Maturity Model Integration (CMMI) [2] is well known as suitable framework to assess software processes; nevertheless the metamodel of CMMI can be extended to evaluate capabilities for change of enterprise and service-oriented software architectures.
3. Software patterns [3], [4], [5], [6] can be applied consistently in architecture quality assessments for both capability diagnostics and for quality improvements of service-oriented architectures for adaptive systems.

In areas where flexibility or agility in business is important, services computing is the approach of choice to organize and utilize distributed capabilities. Innovation oriented companies have introduced in recent years service-oriented architectures (SOA) to assist in closing the business - IT gap by delivering efficiently appropriate business functionality and integrating legacy systems with standard application platforms. Initially SOA was burdened with hype and inflated expectations. The benefits of SOA are recognized for systems on the way to cloud computing and being ready for extended service models. They comprise flexibility, process orientation, time-to-market, and innovation. The adoption of tools and methods for SOA is growing. An overview about the current status of SOA adoption and reports on the maturity of SOA technology from vendors are provided by various analysts (see e.g. [7]). Our approach of investigating the SOA ability [3], [8] of standard platforms in commercial use [9] assembles elements from convergent architecture methods, technologies and related software patterns as in [6], [4], and [5] with service-oriented enterprise architecture management.

The idea of patterns is based on the seminal work of [10] and the specific software patterns originated from the work of [6]. According to Alexander a pattern records the architecture decisions taken by many builders in many places over many years in order to resolve a particular problem. Patterns are human readable structures of text and graphics showing a standardized and repeatable way to derive a solution from a specified problem in a specific context. Our software quality pattern language for architecture assessments consists of 43 integrated patterns, which are used to assess different quality aspects and specific structures of enterprise software architectures.

To analyze the SOA ability of major vendor platforms in a systematic way, the SOA Innovation Lab has developed a questionnaire-based assessment method based on a specifically designed SOA architecture maturity framework. The SOAMMI architecture maturity framework [11] was constructed by integrating different analysis approaches for architecture dimensions, using a consistent metamodel and was based on correlation analysis of specific intrinsic model elements from related state of art maturity frameworks. In particular we use TOGAF [1] as a basic structure for enterprise architecture, spanning all relevant levels.

Details about the enterprise software architecture reference model ESARC (Section II) and the related SOAMMI framework with suitable quality diagnostic patterns, the corresponding questionnaire and findings from consecutive assessments with four major vendors are focus of this paper. In the following Section II we provide our original developed ESARC architecture reference model, which relates to current standards [8] and [1]. On the base of ESARC we provide in Section III our model synthesis for SOA Maturity Model Integration (SOAMMI) (based on current extensions of our previous work from [11], [12]) by assembling and shifting basic maturity model elements into

our conceptual model for architecture maturity diagnostics. Based on the metamodel of SOAMMI, Section IV presents examples from our pattern language (which is based on our previous researched pattern catalog [13]) for diagnostics and optimization of service-oriented enterprise architectures. Section V states our conclusions and validation results from a major sequence of practical assessments with four global software technology vendors and presents some ideas from current research and for future work.

II. ENTERPRISE SOFTWARE ARCHITECTURE REFERENCE MODEL

ESARC – the Enterprise Software Architecture Reference Cube - is our original abstract architecture reference model which provides an integral view for main interweaved architecture types, derived primarily from state of art architecture frameworks like TOGAF [1], essential [14], and the service model of ITIL, and from state of art resources for service-oriented computing [3], [8]. ESARC abstracts from a concrete business scenario or technologies. The Open Group Architecture Framework (TOGAF) [1] is the current standard for enterprise architecture and provides the basic blueprint and structure for our extended service-oriented enterprise software architecture domains like: Architecture Governance, Architecture Management, Business & Information Architecture, Information Systems Architecture, Technology Architecture, Operation Architecture, and the Service Architecture.

The ESARC – Enterprise Software Architecture Reference Cube in Figure 1 unifies as a reference model these architecture domains and provides a coherent aid for examination, comparison, classification and quality ratings of different architecture aspects.

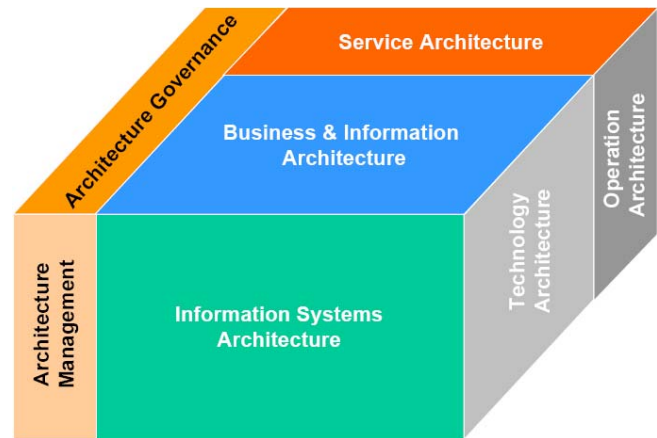


Figure 1. ESARC - Enterprise Software Architecture Reference Cube

The main types of enterprise software architectures like Business & Information Architecture, the Information Systems Architecture, and the Technology Architecture are organized by the Architecture Governance and Management

framework. Architecture Governance defines and maintains the Architecture Governance cycle. The Architecture Governance cycle sets the abstract governance frame for concrete architecture activities within the enterprise or a product line development and specifies the following management activities: plan, define, enable, measure, and control. The second aim of Architecture Governance is to set rules for architecture compliance with internal and external standards. In addition policies for governance and decision definition are set to allow a standardized and efficient process for architecture decisions within the enterprise architecture organization. Enterprise and software architects are acting on a sophisticated connection path coming from business and IT strategy to the architecture landscape realization for interrelated business domains, applications and technologies. Architecture Governance has to set rules for the empowerment of people, defining the structures and procedures of an Architecture Governance Board, and setting rules for communication. Benefits of well organized architecture governance [1] are: transparency of accountability, controlled risk management, protection of the existing asset base through maximizing re-use of existing architectural components, proactive control, monitoring, and management mechanisms, re-use of architectural assets across organizational business units, value creation through monitoring, measuring, evaluation, and feedback, increased visibility of decision-making, greater shareholder value, and integration with existing methodologies and processes by adding control capabilities. With specifications from Architecture Governance we define our main ESARC Architecture Management (in adaption of [1]) procedures for service-oriented enterprise software architectures: service strategy and life cycle management of software and system architecture artifact's state, service security, service testing and monitoring, service contracts, registries, service reuse, and service ownership, definition, versioning.

Benefits resulting from well maintained enterprise or product software architecture are [1]: a more efficient IT operation, lower costs for software development support and maintenance, increased portability of applications, improved interoperability, easier system and network management, improved ability to address critical enterprise-wide issues like security, easier upgrade and exchange of system components, better return on investment and reduced risk for future investment, reduced complexity in infrastructure, maximum return on investment in existing IT infrastructure, the flexibility to make, buy, or outsource IT solutions, reduced risk overall in new investment, the costs of IT ownership, faster, simpler, and cheaper procurement, buying decisions are simpler, the procurement process is faster, and the ability to procure heterogeneous, multi-vendor open systems.

The ESARC - Business & Information Reference Architecture defines the link between the enterprise business strategy and the results of supporting strategic initiatives

through information systems. The Business & Information Reference Architecture provides a single source and comprehensive repository of knowledge from which concrete corporate initiatives will evolve and link. This knowledge is model based and defines an integrated enterprise model of the business which includes the organization and business processes. The Business & Information Reference Architecture opens a connection to IT infrastructures, IT systems, and software as well as security architectures. It provides integration capabilities for IT management, software engineering, service & operations management, and process improvement initiatives. The Business & Information Reference Architecture confer the basis for business IT alignment and therefore models the business and information strategy, the organization, and main business requirements for information systems like key business processes, business rules, business products, services, and related business control information.

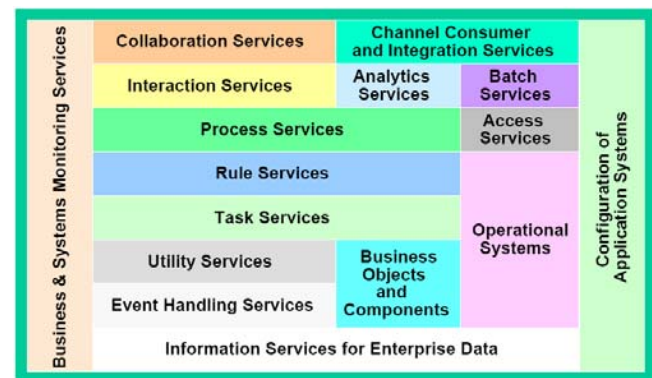


Figure 2. ESARC - Information Systems Reference Architecture

The ESARC – Information Systems Reference Architecture in Figure 2 provides an abstract blueprint of the individual solution architecture for application systems to be deployed and individually customized. The ESARC – Information Systems Reference Architecture contains the main application specific service types and defines their relationship by a layer model of building services. The core functionality of domain services is linked with application interaction capabilities and with the business processes of the customer's organization.

The OASIS Reference Model for Service Oriented Architecture [8] is an abstract framework which guides our ESARC reference architecture. The ESARC provides on his part the abstract model for specific applications architectures and concrete implementations.

We have differentiated a whole sequence of layered service types in our ESARC – Information Systems Reference Architecture. The information services for enterprise data can be thought of as data centric components providing access to the persistent entities of the business process. The capabilities of information services combine both elementary access to CRUD (create, read, update, delete)

operations and complex functionality for finding/searching of data or complex data like data composites or other complex-typed information. Close to the access of enterprise data are context management capabilities provided by the technology architecture: error compensation or exception handling, seeking for alternative information, transaction processing of both atomic and long running and prevalent distributed transactions. Information services and their related data architecture are core company assets and should be close and centrally managed for reuse. Task services implement business capabilities related to specific actions of the business process. Task services could be own or third-party services. Usually task services don't manage state information directly, but work in cooperation with information services. The access to information services follows an acyclic graph according the common control of layer models - from top to down layers.

Operations of task and entity services should not have any knowledge about their process context or interactive usage context. Task service operations should be independent from users and sessions and should only implement business functionality. Authorization checks should be done outside of the business operations. Task and information services should use a transactional context, but their operations shouldn't implement by their own local transactions. Task service operations should be usable both in batch and in online system transactions. Task services are used in process services as multiple composites of services and should therefore be centrally managed high reusable assets. Rule services provide declarative capabilities for adaptable business product and business service structures. Rules provide additional flexible control elements for agile business processes.

Process services are long running services which compose task services and information services into workflows to implement the procedural logic of business processes. Process services can activate rule services to swap out a part of the potentially unstable gateway-related causal decision logic. Process services are frontend by interaction services or by specific diagnostic service and process monitoring services. Often process services manage distributed data and application state indirectly, by activating task and information services. Process services participate in atomic transactions only when they are activated from batch services. When processes services participate in human interaction workflows they have to support long-running transactions where compensation of possible errors or exceptions happens in the business logic.

The ESARC – Technology Reference Architecture in Figure 3 describes the logical software and hardware capabilities that are required to support the deployment of business, data, and application services. This includes IT infrastructure, middleware, networks, communications, processing, and standards. The layers of the ESARC – Technology Reference Architecture and the layers of the

ESARC – Information Systems Reference Architecture correspond to each other.

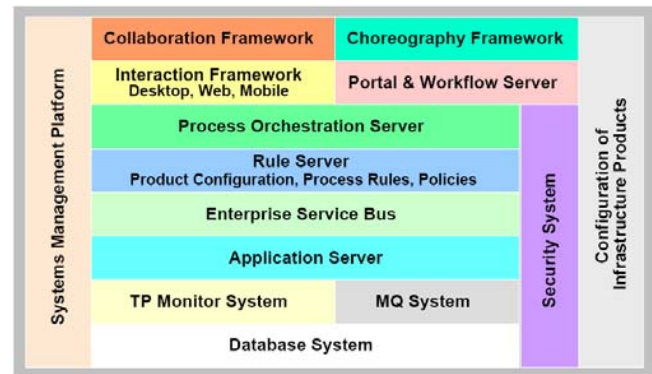


Figure 3. ESARC - Technology Reference Architecture

The database system is the vendor supported database management system for handling enterprise data. Close associated with the database system we have included in our architecture stack the TP-monitor system and optionally a reference model of a messaging system. The application server is the container environment architecture for objects and enterprise components. The enterprise service bus uses a flexible and standard-based messaging mechanism to interconnect services on a process and service execution platform. On top of the service bus we have placed the rule server, which is able to represent business rule and operate rule processing by assembling dynamic inference chains. The process orchestration server executes process services by calling suitable services of earlier mentioned types. For running interaction and collaboration services additional infrastructures are positioned in our stack of the technology reference architecture like: interaction frameworks, portal servers, workflow engines, and nowadays more often collaboration frameworks. Security services are part of an integral framework-based security system of standards and components and are impacted by the mentioned services and by distributed service technologies.

III. SOAMMI – ARCHITECTURE FRAMEWORK

Our architecture reference model relates closely to SOAMMI which is our special designed maturity framework for enterprise and service-oriented product architectures. We have applied our service-oriented Enterprise-Software-Architecture-Reference-Model in several assessment workshops with global vendors of service-oriented platforms. This experience provides the base for further investigations and improvements of our research.

The aim of the original SOAMMI – SOA Maturity Framework Integration - was developed to provide a holistic framework to assess the quality of service-oriented

enterprise architectures. The development process of SOAMMI consisted of two interwoven phases:

- First, CMMI [2] was transformed from an assessment framework for software processes into a specific framework [12] to diagnose systematically architecture capabilities and the overall maturity of enterprise and software architectures.
- Second, we used assessment criteria, maturity domains, architecture capabilities, and level rankings from state of art SOA maturity models, as described in [15] [16] [17] [18]. In addition to generic architecture elements from [1] we adapted specific architecture elements from [14] to develop our service-oriented architecture maturity model.

Our maturity assessment approach was performed by SOA users, having experience in specific business domains and analyzing SOA vendor products for heterogeneous environments of legacy and standard applications.

Our metamodel introduces in Figure 4 maturity levels and defines important quality criteria for service-oriented software and enterprise architecture to measure the architecture maturity of vendor products in respect to requirements from customer oriented domain models and reference use scenarios.

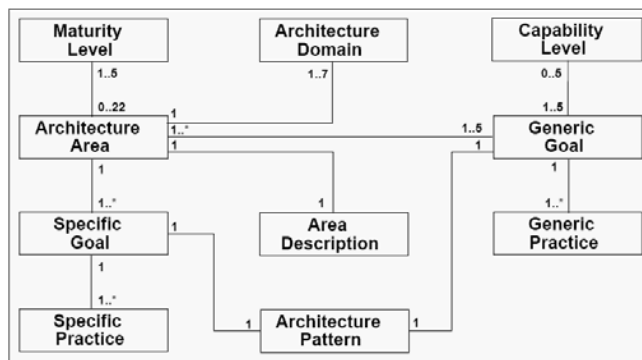


Figure 4. SOAMMI Metamodel

The model contains five maturity levels [2], [11], [12]. They characterize the maturity of a service-oriented organization as a whole, including all areas of enterprise architecture management. Currently most service-oriented platforms are expected to be placed within level one and three of total 5 maturity levels. The main characteristics of those are:

Maturity Level 1: Initial

- Vendor service architecture is not performed or contains initial coverage only
- Architecture is unpredictable and poorly controlled
- Initial service architecture methods and corresponding knowledge transfer only

Maturity Level 2: Managed

- Vendor service architecture is managed, with medium completeness and coverage

- Vendor supports architecture enablement, corrective actions when necessary
- Vendor service architecture is institutionalized within own products

Maturity Level 3: Defined

- Vendor service architecture is defined with large completeness and coverage
- Customer service architecture is agile, derived from standard vendor architecture
- Vendor supports service strategy and architecture governance with methods, tools

A. Architecture Domains and Areas

Having the mentioned understanding of quality criteria associated with maturity levels, we derived the architecture domains from TOGAF [1], where they are used as specific architecture subtypes and corresponding phases of the ADM (Architecture Development Method). The top level structure of SOAMMI is organized by orthogonal architecture domains, mentioned in Section II. Architecture areas are correspondent parts of process areas from CMMI [2]. We have defined 22 specific architecture areas of SOAMMI, fitting our architecture evaluation scope, but different from CMMI, and structured them according to standard architecture maturity levels, being in line with the mentioned architecture domains. Each architecture area is accurately described by name, identification, and a detailed description.

B. Examples of Architecture Areas

Next examples are similarly to process area descriptions of CMMI and show two specific architecture areas of the Business & Information Architecture Domain, using a standardized form for descriptive structures and associated specific goals and specific practices:

1. Architecture Area: BPS Business Products & Services

Purpose: Structure, design, model, and represent business products and associated business services, which are necessary to support modeled products.

Maturity Level: 3

Specific Goals (SG) and Specific Practices (SP):

- SG 1: Model Business Products as Origin of Business Processes
 - SP 1.1 Structure business products within product lines
 - SP 1.2 Design business products by defining product structures and product rules
 - SP 1.3 Model and represent business products
- SG 2: Model Business Services associated with Business Products

- SP 2.1 Structure business services according product types
- SP 2.2 Design business services by defining service structures and service levels
- SP 2.3 Model and represent business services

2. *Architecture Area: BPR Business Processes & Rules*

Purpose: Structure, design, model, and represent business value chains and business processes to support modeled products and services.

Maturity Level: 2

Specific Goals (SG) and Specific Practices (SP):

- SG 1: Model Business Value Chains as Root of Business Processes
 - SP 1.1 Identify business value for business operations
 - SP 1.2 Structure value chains
 - SP 1.3 Optimize business considering customer channels and supplier networks
- SG 2: Model and Optimize Business Processes
 - SP 2.1 Identify business activities for business processes: system activities, user interaction activities, manual activities
 - SP 2.2 Structure business processes for business roles and organizational units
 - SP 2.3 Define business workflows and business process rules
 - SP 2.4 Model and represent business processes
- SG 3: Model and Represent Business Control Information
 - SP 3.1 Identify and represent control information for product monitoring
 - SP 3.2 Identify and represent control information for process monitoring.

IV. ARCHITECTURE QUALITY PATTERNS

Our ESA-Pattern-Language for architecture quality assessments of service-oriented adaptive systems consists of a set of methods which uses best practices for diagnosing malfunctions and for improving the quality of enterprise and information systems architectures. We have derived pattern-based quality methods from the structures of the metamodel of SOAMMI from Section III. Patterns, as described originally by Alexander [10] are collections of best practices which are based on core causalities for problem solving, starting with a description of a recurring problem directing us to a standardized and well proved solution. Complementary to core causalities for problem solving each pattern approach has added important but divergent extensions, resulting in specific canonical forms for software patterns (see examples of different pattern forms in [4], [3], [5], and [6]).

The developed ESA-Pattern-Language (extends our seminal work on a related pattern catalog from [13]) for quality assessments of enterprise software architectures follows the SOAMMI metamodel and organizes the collection of patterns according to the following structures: Architecture Domains, Architecture Areas, Problem Description - associated with specific goals, Solution Elements - connected to relate specific practices, and Related Patterns - connections to next applicable patterns of the pattern language.

Linking solution elements to specific practices of the SOAMMI framework enables concrete solutions for diagnostics and improvement of service-oriented enterprise architectures. This diagnostic and improvement knowledge is no design knowledge, it is rather a procedural assessment knowledge, which is based on standards, best practices, and assessment experience for quality of software and enterprise architectures. It is therefore both concrete and specific for setting the status of service-oriented enterprise architectures, and helping to establish an improvement path for change.

Patterns of our language show what to assess. Our patterns aim to represent diagnostic and improvement procedural knowledge to support cooperative assessment and improvement work of many people over long terms in cyclic assessments of service-oriented enterprise architectures.

Associated with this pattern language we have set up an assessment process showing how to assess architecture capabilities. This process is based on a questionnaire for assessment workshops providing concrete questions and answer types, and helping to direct and standardize the related assessment process. Additionally, we have included process methods for workshops, result evaluations, improvement path information for technology vendors and for application organizations, as well as change support and innovation monitoring instruments.

Based on the two examples of architecture areas from Section II - BPS Business Products & Services Architecture and BPR Business Process & Rules - we are deriving exemplarily the related subset of architecture patterns from the mentioned specific goals:

1. Model Business Products as Origin of Business Processes,
2. Model Business Services associated with Business Products,
3. Model Business Value Chains as Root of Business Processes,
4. Model and Optimize Business Processes, and
5. Model and Represent Business Control Information.

We have chosen the most succinct representation of the core causalities of our diagnostic and improvement patterns. Our pattern form denominates consciously the problem and the solution part as basic elements of our diagnostic and improvement patterns for service-oriented enterprise architectures. To handle complexity, we have reduced the syntax of our architecture quality patterns to essential structures and define our reduced canonical form by the

following elements: name of the pattern, problem to be solved by the pattern, solution elements to solve the problem, and related patterns as connective elements of our pattern language.

This basic canonical form is extendable in further work by additional parts like contexts, forces, examples, explanations, linked patterns, and others. We note that our diagnostic and improvement patterns are basically process patterns for enterprise architecture management and are therefore not as fine granular as classical design patterns.

The following examples show a concrete extract of 5 related patterns – as a pattern language scenario - from our set of 38 architecture diagnostics and improvement patterns:

1. *Pattern Example: Business Product*

Problem: How can we structure, design, model, and represent each business product as an origin for modeling business processes?

Solution:

- Structure business products within product lines
- Design business products by defining product structures and product rules
- Model and represent business products

Related Patterns: Business Service, Value Chain, Business Process, Business Control Information

2. *Pattern Example: Business Service*

Problem: How can we structure, model, and represent each business service needed to support business products?

Solution:

- Structure business services according product types
- Design business services by defining service structures and service levels
- Model and represent business services

Related Patterns: Value Chain, Business Process, Business Control Information

3. *Pattern Example: Value Chain*

Problem: How can we structure, optimize and represent business value chains as roots for business process modeling?

Solution:

- Identify business value for business operations
- Structure value chains
- Optimize business considering customer channels and supplier networks

Related Patterns: Business Process

4. *Pattern Example: Business Process*

Problem: How can we structure, model and optimize business processes, related workflows, and business process rules?

Solution:

- Identify business activities for business processes: system activities, user interaction activities, manual activities

- Structure business processes for business roles and organizational units
- Define business workflows and business process rules
- Model and represent business processes

Related Patterns: Business Control Information

5. *Pattern Example: Business Control Information*

Problem: How can we model and represent business monitoring and control information?

Solution:

- Identify and represent control information for product monitoring
- Identify and represent control information for process monitoring

Related Patterns: None

The mentioned singular pattern examples are part of a scenario of pattern interactions. This scenario is specified by a graph of architecture quality patterns, which defines the structure of our architecture quality pattern language.

The basic causality of our architecture quality patterns allows us to navigate in two directions (Figure 5): From the problem statement to the solution and backwards from the expected solution to the problem.

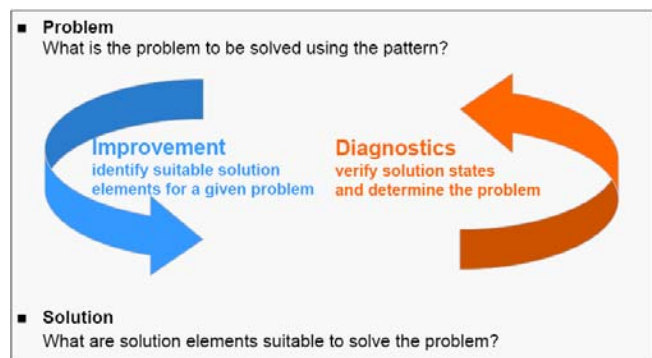


Figure 5. Architecture Diagnostics and Improvement Patterns

The navigation possibilities within an architecture quality pattern enable two important problem solving strategies:

1. Diagnostics: for verifying suggested solutions and defining the problem (from pattern solution to the pattern problem statement)
2. Improvement: for identifying suitable solution elements for a given problem (from the pattern problem statement to the solution statements).

The practical benefit of our architecture quality pattern language is documented by the successful use as guideline for questionnaire design for architecture capability assessments of four major vendors: IBM, SAP, ORACLE, and Microsoft. During the validation of the ESARC, SOAMMI and the related ESA-Pattern-Language we have consolidated user requirements and mapped them against specific goals. In addition to the answers to all specific questions, we have synthesized key findings that highlight

our view on the actual SOA ability of a standard platform across vendors.

Even though SOA has been a topic for vendors for years now, there are no major SOA implementations that include standard software systems. Most cases have the quality of a proof of concept, often focusing on GUI integration, instead of deep functional integration. There seems to be a gap between those SOA capabilities that are offered and those which can be actually used in a SOA. In summary, there are still obstacles to apply standard software in a heterogeneous SOA environment. Often, a vendor's SOA approach is specific to the vendor. Vendors also often use specific semantics and data models and have incompatible technologies (ESB, repository) that do not integrate seamlessly into overall heterogeneous landscapes.

V. CONCLUSION

Our original approach for architecture evaluation and optimization of service-oriented enterprise software architectures is based on a special architecture reference model, an associated architecture metamodel and an architecture pattern language. In our research we have motivated the necessity to extend both existing architecture reference models and service-oriented maturity models to accord to a clear metamodel approach due to the well understood and verified CMMI model. Our approach provides a sound basis from theory for practical evaluations of service oriented standard platforms in heterogeneous environments with four major global acting technology vendors. The ESARC, the SOAMMI framework and the related architecture quality ESA-Patterns were applied in several assessment workshops with vendors of service-oriented platforms and have provided transparent results for subsequent changes on service-oriented product architectures and related processes. It should be noted however, that the results of these assessments need to be interpreted in the context of company specific strategies and use cases. As a consequence they cannot provide vendor rankings of any kind.

Future work has to consider conceptual work on both static and dynamic architecture complexity, and in connecting architecture quality procedures with prognostic processes on architecture maturity with simulations of enterprise and software architectures. Additional improvement idea deals with patterns for visualization of architecture artifacts and architecture control information to be operable on an architecture management cockpit. To improve semantic-based navigation within the complex space of EAM-visualization and service-oriented enterprise software architecture management we are currently working on ontology models for the ESARC – The Enterprise Software Architecture Reference Cube.

ACKNOWLEDGMENT

Many thanks to the SOA Innovation Lab www.soa-lab.de initiative, a major business innovation network of service-oriented software technology and application companies in Germany and Europe, for providing us the context for our research and the validation of our results.

REFERENCES

- [1] TOGAF "The Open Group Architecture Framework" Version 9, The Open Group, 2009.
- [2] CMMI-DEV-1.3 2010 "CMMI for Development, Version 1.3," Carnegie Mellon University, Software Engineering Institute, 2010.
- [3] T. Erl, "SOA Design Patterns", Prentice Hall, 2009.
- [4] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, M. Stal, „Pattern-oriented Software Architecture“, Wiley, 1996.
- [5] M. Fowler, "Patterns of Enterprise Application Architecture," Addison Wesley, 2003.
- [6] E. Gamma, R. Helm, R. Johnson, J. Vlissides, "Design Patterns", Addison Wesley, 1994.
- [7] F.L. Kenney, D.C.Plummer, "Magic Quadrant for integrated SOA Governance Technology Sets," Gartner Research 2009.
- [8] OASIS "Reference Model for Service Oriented Architecture" 1.0, 2006.<http://www.oasis-open.org/committees/download.php/19679/soa-rm-cs.pdf>,
- [9] H. Buckow, H.-J. Groß, G. Piller, K. Prott, J. Willkomm, A. Zimmermann, "Method for Service-Oriented EAM with Standard Platforms in Heterogeneous IT Landscapes". Proc. 2nd European Workshop on Patterns for Enterprise Architecture Management (PEAM2010), Paderborn, GI-Edition - (LNI), P-160, 2010, pp. 219-230.
- [10] C. Alexander, S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, S. Angel, "A Pattern Language", Oxford University Press New York, 1977.
- [11] A. Zimmermann, "Method for Maturity Diagnostics of Enterprise and Software Architectures", A. Erkollar (Ed.) ENTERPRISE & BUSINESS MANAGEMENT, A Handbook for Educators, Consultants and Practitioners, Volume 2, Tectum 2010, ISBN 978-3-8288-2306-8, 2010, pp. 129-172.
- [12] A. Zimmermann, H. Buckow, H.-J. Groß, "SOAMMI – SOA Maturity Model Integration - Conceptual Framework," Version 2, SOA Innovation Lab, www.soa-lab.de, 2010.
- [13] A. Zimmermann, E. Ammann, F. Laux, "Pattern Catalog for Capability Diagnostics and Maturity Evaluation of Service-oriented Enterprise Architectures", PATTERNS 2010 - The Second International Conferences on Pervasive Patterns and Applications, November 21-26, 2010 - Lisbon, Portugal, IARIA Proceedings of the PATTERNS 2010 Conference, 2010, pp. 13-19.
- [14] "The Essential Architecture Project," <http://www.enterprise-architecture.org>
- [15] ACMM, "Architecture Capability Maturity Model", in TOGAF Version 9, The Open Group Architecture Framework, The Open Group, 2009, pp. 685-688.
- [16] S. Inaganti, S. Aravamudan, "SOA Maturity Model" BP Trends, April 2007, 2007, pp. 1-23.
- [17] Oracle: "SOA Maturity Model", <http://www.scribd.com/doc/2890015/oraclesoamaturitymodelcheatsheet>
- [18] Sonic: "A new Service-oriented Architecture (SOA) Maturity Model," http://soa.omg.org/Uploaded%20Docs/SOA/SOA_Maturity.pdf